

# Dataset Description

## Dataset 1: DAIGT V2 Train Dataset (Kaggle)

---

**Total rows:** 44,868 essay samples (commonly cited as 44,800–44,868 depending on the exact CSV version)

**Classes:** 2 (binary classification) label column: 0 = human-written, 1 = AI-generated

**Class balance:** 27,371 human-written vs 17,497 AI-generated (roughly 61% human / 39% AI imbalanced, mimicking real-world proportions)

**Key columns:**

- text — the essay content
- label — 0/1 binary target
- prompt\_name — which essay topic/prompt it responds to (e.g., "Car-free cities," "Driverless cars," "Exploring Venus")
- source — origin of the text (e.g., persuade\_corpus, chat\_gpt\_moth, llama2\_chat, falcon\_180b, darragh\_claude, radek\_500, kingki19\_palm)
- RDizzl3\_seven — flag marking whether the row belongs to the 7 prompts known to overlap with the competition's hidden test set

**AI sources represented:** ChatGPT (GPT-3.5), GPT-4, LLaMA-2-70B, Falcon-180B, PaLM (Google), and 2,000 Claude-generated essays

**Human data:** primarily from the PERSUADE 2.0 corpus (student-written argumentative/expository essays)

**Text type:** short-to-medium academic/persuasive essays, single-topic

**License:** MIT for newly contributed data; underlying sources retain their original licenses

## Dataset 2: HC3 (Human ChatGPT Comparison Corpus)

---

**Purpose:** First large-scale corpus directly comparing human vs. ChatGPT answers the same questions, for detection research (paper: How Close is ChatGPT to Human Experts? arXiv 2301.07597).

**Total rows:** 24,322 (per HF dataset viewer) often cited as 24.3k question-answer pairs in the "all" config since each row holds both a human and a ChatGPT answer as parallel sequences

**Classes:** 2 (binary) humans vs. ChatGPT, though it's structured as paired Q&A rather than pre-labeled rows; you generate labels by splitting human\_answers vs chatgpt\_answers

**Languages:** English and Chinese

**Subsets (by domain), all within "all" config:**

- reddit\_eli5: 17.1k rows (largest subset, from Reddit's "Explain Like I'm Five")
- finance: 3.93k rows
- medicine: 1.25k rows
- open\_qa: 1.19k rows
- wiki\_csai: 842 rows (Wikipedia + computer science/AI questions)

- (a Chinese-language medical subset also exists in the broader HC3 release)

**Key columns:**

- id
- question — the prompt/question
- human\_answers — list of one or more human responses
- chatgpt\_answers — list of one or more ChatGPT responses
- source — which subdomain (eli5, finance, medicine, etc.)

**Class balance:** about 69% human vs. 31% ChatGPT (58,546 human answers vs 26,903 ChatGPT answers) (since each question typically has both answers)

**Text type:** long-form Q&A responses across varied domains (contrasts with DAIGT's essay format)

**License:** CC-BY-SA-4.0 (or stricter, inheriting from source datasets)

## Project Implementation Detail

### ■ Import libraries and download NLTK resources

#### Description:

All imports are placed within one cell so no one on the team is looking for a missing package to go through the notebook. Along with the libraries, the cell pulls down the NLTK data files used later by the tokenizer, the stop-word list and the lemmatizer. The print at the end was mostly added because Colab cells can run without any output and we needed something to show that they actually worked.

#### Code:

```
import re
import pandas as pd
import numpy as np
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer,
TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

#### Sample output:

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True
```

#### Code description:

[Type here]

pandas and numpy handle the tables, re gives us the cleaning regex, and nltk covers tokenization, stop words, and lemmatization. sklearn brings in both vectorizers, all three classifiers, and every metric we needed. matplotlib doesn't get used until much later, for the confusion matrices.

## ■ Extract Dataset from Google Drive Links

### Description:

Both datasets were too big to keep re-uploading every time we opened Colab, so we hosted them on Drive instead and just point pandas at the direct download links.

SAMPLE\_PER\_CLASS also lives in this cell since the balancing step further down depends on it. We settled on 3000 per class after trying a bigger number first and watching training slow to a crawl.

### Code:

```
SAMPLE_PER_CLASS = 3000
path_daigt_download =
'https://drive.google.com/uc?export=download&id=1L1a5lPNUSjmLA167XT3Yv-
S55d9FaW0U'
path_hc3_download =
'https://drive.google.com/uc?export=download&id=1Dse8l3ct3elof5gqvVWcl0
_Wfm38RsV0'
print('Google Drive download links for DAIGT V2 and HC3 datasets are
configured.')
```

### Sample output:

```
Google Drive download links for DAIGT V2 and HC3 datasets are configured.
```

### Code description:

SAMPLE\_PER\_CLASS controls how big our balanced sample ends up being. pandas can read straight from a Drive URL the same way it reads a local CSV, which saved us from mounting Drive or downloading anything by hand.

## ■ Read Dataset 1: DAIGT V2 Train Dataset

### Description:

DAIGT V2 loads directly from the link above. We kept only text and label since that's all the classifier needs; the other three columns weren't relevant here. A quick summary prints below so we could check the numbers before moving on.

### Code:

```
daigt_raw = pd.read_csv(path_daigt_download)
```

[Type here]

```

df1 = daigt_raw[['text', 'label']].dropna()
df1['label'] = df1['label'].astype(int)

print("=" * 60)
print("Dataset 1: DAIGT V2 Train Dataset")
print("Dataset Summary")
print("=" * 60)

print(f"Total Samples : {len(df1):,}")
print(f"Total Columns : {df1.shape[1]}")

print("\nClass Distribution")
print(df1['label']
      .value_counts()
      .sort_index()
      .rename_axis("Label")
      .reset_index(name="Number of Samples"))

print("\nFirst 5 Records")
print(df1.head())

```

### Sample output:

```

=====
Dataset 1: DAIGT V2 Train Dataset
Dataset Summary
=====
Total Samples : 44,868
Total Columns : 2

Class Distribution
   Label  Number of Samples
0      0             27371
1      1             17497

First 5 Records

```

|   | text                                              | label |
|---|---------------------------------------------------|-------|
| 0 | Phones\n\nModern humans today are always on th... | 0     |
| 1 | This essay will explain if drivers should or s... | 0     |
| 2 | Driving while the use of cellular devices\n\nT... | 0     |
| 3 | Phones & Driving\n\nDrivers should not be able... | 0     |
| 4 | Cell Phone Operation While Driving\n\nThe abil... | 0     |

### Code description:

read\_csv grabs the file straight from the URL. Dropping everything except text and label keeps the frame lean, dropna clears out incomplete rows and astype(int) makes sure label is a real integer instead of a string. The prints report row count, column count, class split and the first five records so we could see the data early on.

.

[Type here]

## ■ Read Dataset 2: HC3 (Human ChatGPT Comparison Corpus)

### Description:

HC3 is structured very differently from DAIGT. Instead of one row per text, each record bundles a question with a list of human answers and a list of ChatGPT answers. We had to explode both lists into individual rows before any of it could go near a classifier. This one took a bit more trial and error than we expected.

### Code:

```
hc3_raw = pd.read_json(path_hc3_download, lines=True)

human = (
    hc3_raw[['human_answers']]
    .explode('human_answers')
    .rename(columns={'human_answers': 'text'})
)
human['label'] = 0

chatgpt = (
    hc3_raw[['chatgpt_answers']]
    .explode('chatgpt_answers')
    .rename(columns={'chatgpt_answers': 'text'})
)
chatgpt['label'] = 1

df2 = pd.concat([human, chatgpt], ignore_index=True).dropna()
df2['text'] = df2['text'].astype(str)

print("=" * 60)
print("Dataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print("Dataset Summary")
print("=" * 60)

print(f"Total Samples : {len(df2):,}")
print(f"Total Columns : {df2.shape[1]}")

print("\nClass Distribution")
print(df2['label'].value_counts().sort_index().rename_axis("Label").reset_index(name="Number of Samples"))

print("\nFirst 5 Records")
print(df2.head())
```

### Sample output:

```
=====
Dataset 2: HC3 (Human ChatGPT Comparison Corpus)
Dataset Summary
=====
Total Samples : 85,449
```

[Type here]

Total Columns : 2

Class Distribution

|   | Label | Number of Samples |
|---|-------|-------------------|
| 0 | 0     | 58546             |
| 1 | 1     | 26903             |

First 5 Records

|   | text                                              | label |
|---|---------------------------------------------------|-------|
| 0 | Basically there are many categories of " Best ... | 0     |
| 1 | If you 're hearing about it , it 's because it... | 0     |
| 2 | One reason is lots of catagories . However , h... | 0     |
| 3 | salt is good for not dying in car crashes and ... | 0     |
| 4 | In Minnesota and North Dakota , they tend to u... | 0     |

### Code description:

read\_json with lines=True reads the JSON-lines file. explode is doing the heavy lifting here, turning each list of answers into its own row. We renamed both frames to share a text column, labeled human answers 0 and ChatGPT answers 1, then concatenated them. One thing that caught us off guard: dropna ends up removing 455 rows, because that many questions in the raw data have no ChatGPT answer at all, and explode leaves an empty row behind for each one. After all that, we're left with 85,449 labeled texts.

### ■ Balance both datasets by sampling equal rows per class

#### Description:

Both sets of data are not balanced. DAIGT leans human. HC3 is even more humanistic. We didn't want our models to simply learn "predict the majority class", so we sampled equal number of rows from each label of both datasets.

#### Code:

```
def balance(df, n, seed=42):
    parts = []
    for value in sorted(df['label'].unique()):
        subset = df[df['label'] == value]
        parts.append(subset.sample(n=min(n, len(subset)),
random_state=seed))
    return pd.concat(parts).sample(frac=1,
random_state=seed).reset_index(drop=True)

df1 = balance(df1, SAMPLE_PER_CLASS)
df2 = balance(df2, SAMPLE_PER_CLASS)

print("=" * 50)
print("Dataset 1: DAIGT V2 Train Dataset")
print("Class Distribution After Balancing")
print("=" * 50)
print(df1['label'].value_counts().sort_index().rename_axis("Label").res
et_index(name="Number of Rows"))
```

[Type here]

```

print("\n" + "=" * 50)
print("Dataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print("Class Distribution After Balancing")
print("=" * 50)
print(df2['label'].value_counts().sort_index().rename_axis("Label").reset_index(name="Number of Rows"))

```

### Sample output:

```

=====
Dataset 1: DAIGT V2 Train Dataset
Class Distribution After Balancing
=====
   Label  Number of Rows
0       0             3000
1       1             3000

=====
Dataset 2: HC3 (Human ChatGPT Comparison Corpus)
Class Distribution After Balancing
=====
   Label  Number of Rows
0       0             3000
1       1             3000

```

### Code description:

The function grabs the same number of rows from each label, combines them, and shuffles with `sample(frac=1)`. We fixed `random_state` so the split is reproducible across runs. Both datasets end up at 6,000 rows total, split evenly between classes

- **Preprocess text (case folding, punctuation removal, tokenization, stop word removal, lemmatization)**

### Description:

The vectorizers work better on something consistent as the raw text is messy. We are using 5 steps: case folding, removal of punctuation and other non-letters, tokenization, stop-word removal, and lemmatization.

### Code:

```

lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

def preprocess(text):
    text = str(text).lower()
    text = re.sub(r'^a-z\s', ' ', text)
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(tok) for tok in tokens if tok not in
stop_words and len(tok) > 1]
    return ' '.join(tokens)

```

[Type here]



```
df1['clean_text'] = df1['text'].apply(preprocess)
df2['clean_text'] = df2['text'].apply(preprocess)

print('Sample of Preprocessed Text for Dataset 1 (DAIGT V2):')
print(df1[['text', 'clean_text']].head())
print('\nSample of Preprocessed Text for Dataset 2 (HC3):')
print(df2[['text', 'clean_text']].head())
```

### Sample output:

```
Sample of Preprocessed Text for Dataset 1 (DAIGT V2):
      text \
0  Dear TEACHER_NAME,\n\nI think the idea of havi...
1  Hey there!  So, you wanna know about how havin...
2  Generic_Name\n\nGeneric_Name\n\nEnglish 8 Hono...
3  The school bus doors close with a hiss behind ...
4  It is no secret that cars have become a staple...

      clean_text
0  dear teacher name think idea principal making ...
1  hey wan na know positive attitude lead success...
2  generic name generic name english honor asking...
3  school bus door close hiss behind walk drivewa...
4  secret car become staple american life use get...

Sample of Preprocessed Text for Dataset 2 (HC3):
      text \
0  20 second solver here . We solve the first two...
1  Sure! Declension in German refers to the way t...
2  Hi there, as you had fall 2 weeks back and I s...
3  The side has been playing international rugby ...
4  Microwave presets like "popcorn" or "plate of ...

      clean_text
0  second solver solve first two layer intuition ...
1  sure declension german refers way noun pronoun...
2  hi fall week back suppose neurological issue l...
3  side playing international rugby since british...
4  microwave presets like popcorn plate food prog...
```

### Code description:

The case folding is taken care of by `lower()`. The regex removes punctuation, digits and anything else that is not a letter; `word_tokenize` breaks the cleaned string into tokens; the `list comprehension` removes stop words and single character leftovers in the same pass; `lemmatize` reduces what remains to its base form. We had to concatenate everything at the end, as the vectorizers are expecting text not a list of tokens.

### ■ Train & Test split for both datasets

#### Description:

20% of each data set was reserved for testing. None of that gets touched during training, that's the whole point of a test set.

#### Code:

[Type here]

```

X1_train, X1_test, y1_train, y1_test = train_test_split(
    df1['clean_text'], df1['label'], test_size=0.2, random_state=42,
    stratify=df1['label'])

X2_train, X2_test, y2_train, y2_test = train_test_split(
    df2['clean_text'], df2['label'], test_size=0.2, random_state=42,
    stratify=df2['label'])

print(f'Dataset 1 (DAIGT V2) Training set size: {len(X1_train)}
samples')
print(f'Dataset 1 (DAIGT V2) Testing set size: {len(X1_test)} samples')
print(f'Dataset 2 (HC3) Training set size: {len(X2_train)} samples')
print(f'Dataset 2 (HC3) Testing set size: {len(X2_test)} samples')

```

### Sample output:

```

Dataset 1 (DAIGT V2) Training set size: 4800 samples
Dataset 1 (DAIGT V2) Testing set size: 1200 samples
Dataset 2 (HC3) Training set size: 4800 samples
Dataset 2 (HC3) Testing set size: 1200 samples

```

### Code description:

train\_test\_split does the 80/20 split, stratify maintains the class balance, and random\_state makes everything crystal clear so that the same split is always performed when we rerun the notebook. Then we have 4,800 training texts and 1,200 test texts for each dataset.

## ■ Vector Semantics: Bag of Words (BoW) Feature Extraction

### Description:

Bag of Words is our first representation from the two. Each column is representing one word from the vocabulary and the value shows how many times it comes up in a given text. It is simple but it works.

### Code:

```

from sklearn.feature_extraction.text import CountVectorizer
bow1 = CountVectorizer()
X1_train_bow = bow1.fit_transform(X1_train)
X1_test_bow = bow1.transform(X1_test)

bow2 = CountVectorizer()
X2_train_bow = bow2.fit_transform(X2_train)
X2_test_bow = bow2.transform(X2_test)

print("=" * 70)
print("Bag of Words (BoW) Feature Extraction Summary")
print("=" * 70)

```

[Type here]

```

print("\nDataset 1: DAIGT V2 Train Dataset")
print(f"Training Feature Matrix Shape : {X1_train_bow.shape}")
print(f"Testing Feature Matrix Shape  : {X1_test_bow.shape}")
print(f"Vocabulary Size                  : {len(bow1.vocabulary_):,}")

print("\nDataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print(f"Training Feature Matrix Shape : {X2_train_bow.shape}")
print(f"Testing Feature Matrix Shape  : {X2_test_bow.shape}")
print(f"Vocabulary Size                  : {len(bow2.vocabulary_):,}")

```

### Sample output:

```

=====
Bag of Words (BoW) Feature Extraction Summary
=====

Dataset 1: DAIGT V2 Train Dataset
Training Feature Matrix Shape : (4800, 20465)
Testing Feature Matrix Shape  : (1200, 20465)
Vocabulary Size                : 20,465

Dataset 2: HC3 (Human ChatGPT Comparison Corpus)
Training Feature Matrix Shape : (4800, 19599)
Testing Feature Matrix Shape  : (1200, 19599)
Vocabulary Size                : 19,599

```

### Code description:

The vocabulary of the `fit_transform` is built from training data only and then transform applies that vocabulary to the test set. We were careful about this so that the test information would not leak into training. The shapes which were printed confirm the vocabulary size.

## ■ Vector Semantics: TF-IDF Feature Extraction

### Description:

Our second representation is called TF-IDF. It's still counting words, but words that occur in nearly every text are downweighted so that what is left is more likely to differentiate the two classes.

### Code:

```

from sklearn.feature_extraction.text import TfidfVectorizer

tfidf1 = TfidfVectorizer()
X1_train_tfidf = tfidf1.fit_transform(X1_train)
X1_test_tfidf = tfidf1.transform(X1_test)

tfidf2 = TfidfVectorizer()
X2_train_tfidf = tfidf2.fit_transform(X2_train)
X2_test_tfidf = tfidf2.transform(X2_test)

```

[Type here]

```

print("=" * 70)
print("TF-IDF Feature Extraction Summary")
print("=" * 70)

print("\nDataset 1: DAIGT V2 Train Dataset")
print(f"Training Feature Matrix Shape : {X1_train_tfidf.shape}")
print(f"Testing Feature Matrix Shape : {X1_test_tfidf.shape}")
print(f"Vocabulary Size : {len(tfidf1.vocabulary_):,}")

print("\nDataset 2: HC3 (Human ChatGPT Comparison Corpus)")
print(f"Training Feature Matrix Shape : {X2_train_tfidf.shape}")
print(f"Testing Feature Matrix Shape : {X2_test_tfidf.shape}")
print(f"Vocabulary Size : {len(tfidf2.vocabulary_):,}")

```

### Sample output:

```

=====
TF-IDF Feature Extraction Summary
=====

Dataset 1: DAIGT V2 Train Dataset
Training Feature Matrix Shape : (4800, 20465)
Testing Feature Matrix Shape : (1200, 20465)
Vocabulary Size : 20,465

Dataset 2: HC3 (Human ChatGPT Comparison Corpus)
Training Feature Matrix Shape : (4800, 19599)
Testing Feature Matrix Shape : (1200, 19599)
Vocabulary Size : 19,599

```

### Code description:

We used TfidfVectorizer the same way as CountVectorizer, fit on train, apply to test. The vocabulary comes out the same size as before and the stored values differ only.

## ■ Evaluation helper and results container

### Description:

We are training twelve models total across this project and evaluating each one by hand felt like we are doing copy paste type mistakes. So we wrote one helper function instead of evaluating each. It fits the model, predicts and reports accuracy, precision, recall, and F1 in one shot.

### Code:

```

results = {}
print("The 'results' dictionary will store evaluation metrics. The
'evaluate' function trains a model, makes predictions, and calculates
key performance indicators like accuracy, precision, recall, and F1
score.")

```

[Type here]

```
def evaluate(model, X_train, y_train, X_test, y_test, title):
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    acc = accuracy_score(y_test, predictions)
    prec = precision_score(y_test, predictions, average='weighted',
zero_division=0)
    rec = recall_score(y_test, predictions, average='weighted',
zero_division=0)
    f1 = f1_score(y_test, predictions, average='weighted',
zero_division=0)
    print(title)
    print(classification_report(y_test, predictions))
    print('Accuracy:', acc)
    print('-' * 60)
    return {'Accuracy': acc, 'Precision': prec, 'Recall': rec, 'F1':
f1, 'y_true': y_test, 'y_pred': predictions}

print("'evaluate' function defined and ready for use.")
```

### Sample output:

The 'results' dictionary will store evaluation metrics. The 'evaluate' function trains a model, makes predictions, and calculates key performance indicators like accuracy, precision, recall, and F1 score. 'evaluate' function is defined and ready for use.

### Code description:

evaluate() fits the model, predicts on the held-out test set then computes all four metrics. We used weighted averaging for precision, recall, and F1. Though since our classes are balanced that ends up matching the macro average anyway. The function also returns the true and predicted labels so we could reuse them for the confusion matrices later instead of retraining every model twice.

## ■ Naive Bayes (MultinomialNB)

### Description:

Naive Bayes is the first of our three classifiers. We ran it across all four combinations on both representations of both datasets and stored every result for further usage.

### Code:

```
results[('Naive Bayes', 'BoW', 'Dataset 1')] =
evaluate(MultinomialNB(), X1_train_bow, y1_train, X1_test_bow, y1_test,
'Naive Bayes | BoW | Dataset 1 (DAIGT V2)')
results[('Naive Bayes', 'TF-IDF', 'Dataset 1')] =
evaluate(MultinomialNB(), X1_train_tfidf, y1_train, X1_test_tfidf,
y1_test, 'Naive Bayes | TF-IDF | Dataset 1 (DAIGT V2)')
```

[Type here]

```

results[('Naive Bayes', 'BoW', 'Dataset 2')] =
evaluate(MultinomialNB(), X2_train_bow, y2_train, X2_test_bow, y2_test,
'Naive Bayes | BoW | Dataset 2 (HC3)')
results[('Naive Bayes', 'TF-IDF', 'Dataset 2')] =
evaluate(MultinomialNB(), X2_train_tfidf, y2_train, X2_test_tfidf,
y2_test, 'Naive Bayes | TF-IDF | Dataset 2 (HC3)')
print('Naive Bayes model evaluation results stored in the results
dictionary.')

```

### Sample output:

| Naive Bayes   BoW   Dataset 1 (DAIGT V2) |   |           |        |          |         |
|------------------------------------------|---|-----------|--------|----------|---------|
|                                          |   | precision | recall | f1-score | support |
|                                          | 0 | 0.93      | 0.98   | 0.96     | 600     |
|                                          | 1 | 0.98      | 0.93   | 0.96     | 600     |
| accuracy                                 |   |           |        | 0.96     | 1200    |
| macro avg                                |   | 0.96      | 0.96   | 0.96     | 1200    |
| weighted avg                             |   | 0.96      | 0.96   | 0.96     | 1200    |

Accuracy: 0.9566666666666667

---

| Naive Bayes   TF-IDF   Dataset 1 (DAIGT V2) |   |           |        |          |         |
|---------------------------------------------|---|-----------|--------|----------|---------|
|                                             |   | precision | recall | f1-score | support |
|                                             | 0 | 0.93      | 0.97   | 0.95     | 600     |
|                                             | 1 | 0.97      | 0.93   | 0.95     | 600     |
| accuracy                                    |   |           |        | 0.95     | 1200    |
| macro avg                                   |   | 0.95      | 0.95   | 0.95     | 1200    |
| weighted avg                                |   | 0.95      | 0.95   | 0.95     | 1200    |

Accuracy: 0.9508333333333333

---

| Naive Bayes   BoW   Dataset 2 (HC3) |   |           |        |          |         |
|-------------------------------------|---|-----------|--------|----------|---------|
|                                     |   | precision | recall | f1-score | support |
|                                     | 0 | 0.87      | 0.84   | 0.86     | 600     |
|                                     | 1 | 0.85      | 0.88   | 0.86     | 600     |
| accuracy                            |   |           |        | 0.86     | 1200    |
| macro avg                           |   | 0.86      | 0.86   | 0.86     | 1200    |
| weighted avg                        |   | 0.86      | 0.86   | 0.86     | 1200    |

Accuracy: 0.8583333333333333

---

| Naive Bayes   TF-IDF   Dataset 2 (HC3) |   |           |        |          |         |
|----------------------------------------|---|-----------|--------|----------|---------|
|                                        |   | precision | recall | f1-score | support |
|                                        | 0 | 0.90      | 0.74   | 0.81     | 600     |
|                                        | 1 | 0.78      | 0.92   | 0.84     | 600     |
| accuracy                               |   |           |        | 0.83     | 1200    |
| macro avg                              |   | 0.84      | 0.83   | 0.83     | 1200    |
| weighted avg                           |   | 0.84      | 0.83   | 0.83     | 1200    |

Accuracy: 0.8275

[Type here]

Naive Bayes model evaluation results stored in the results dictionary.

### **Code description:**

For each setting MultinomialNB is executed through our shared evaluate() helper. It is a rather crude model of language, in that it assumes that given the class, the probabilities of words are independent of one another which real sentences don't follow. Nevertheless, it trains quickly and provided us with a good reference point for comparing the other two models to it.

#### **■ Confusion matrices for Naive Bayes**

### **Description:**

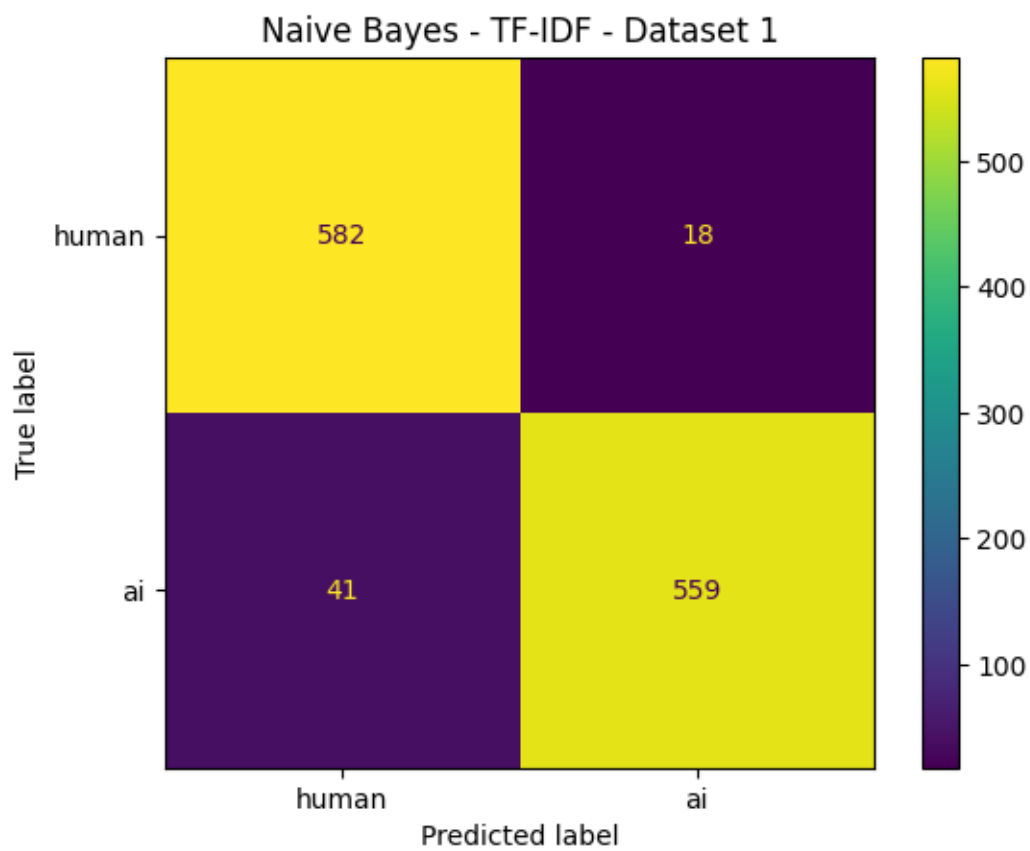
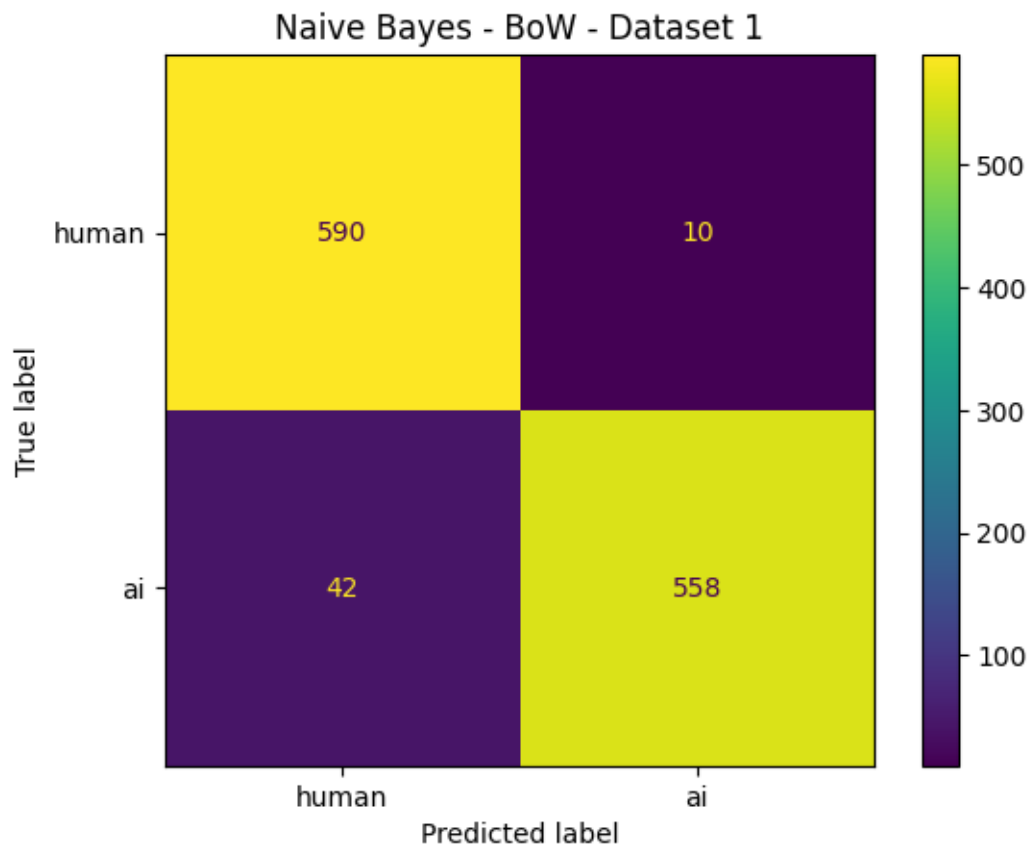
There's a lot of accuracy to be concealed. For each of the four runs of Naive Bayes, we created a confusion matrix to visualize where it was getting confused between human and AI text.

### **Code:**

```
combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW',
'Dataset 2'), ('TF-IDF', 'Dataset 2')]
for rep, ds in combos:
    scores = results[('Naive Bayes', rep, ds)]
    ConfusionMatrixDisplay(confusion_matrix(scores['y_true'],
scores['y_pred']), display_labels=['human', 'ai']).plot()
    plt.title('Naive Bayes' + ' - ' + rep + ' - ' + ds)
    plt.show()
print('Confusion matrices for Naive Bayes models displayed above.')
```

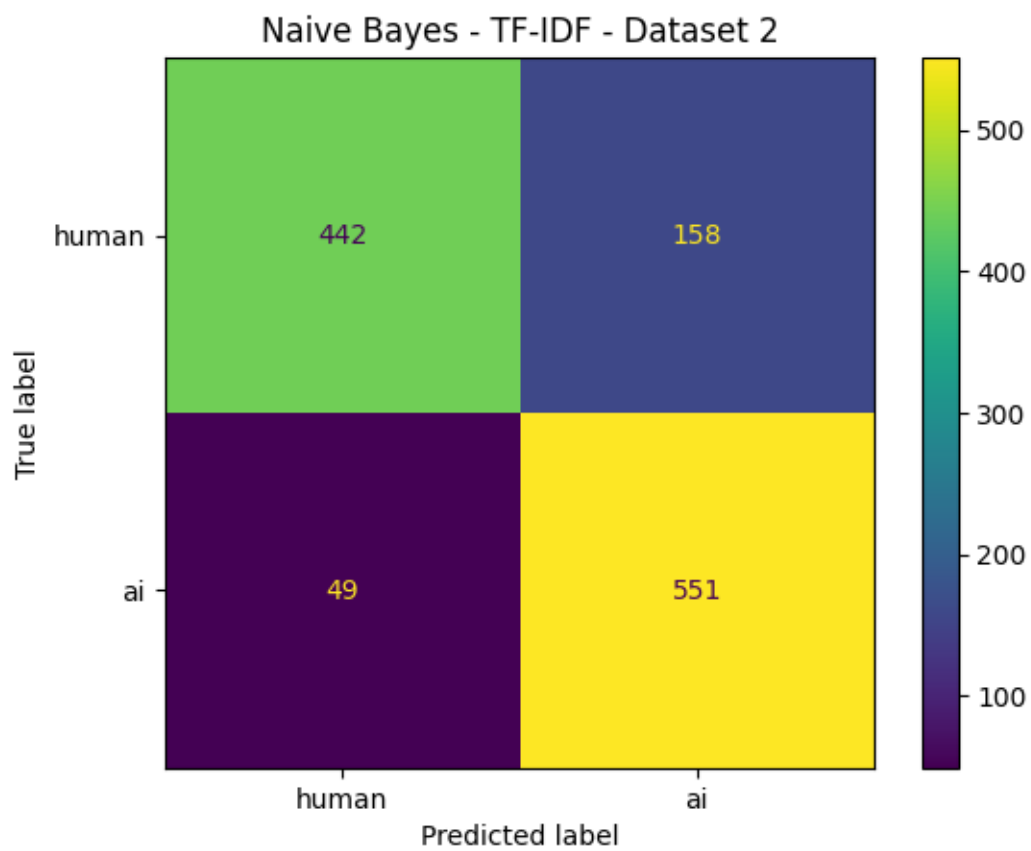
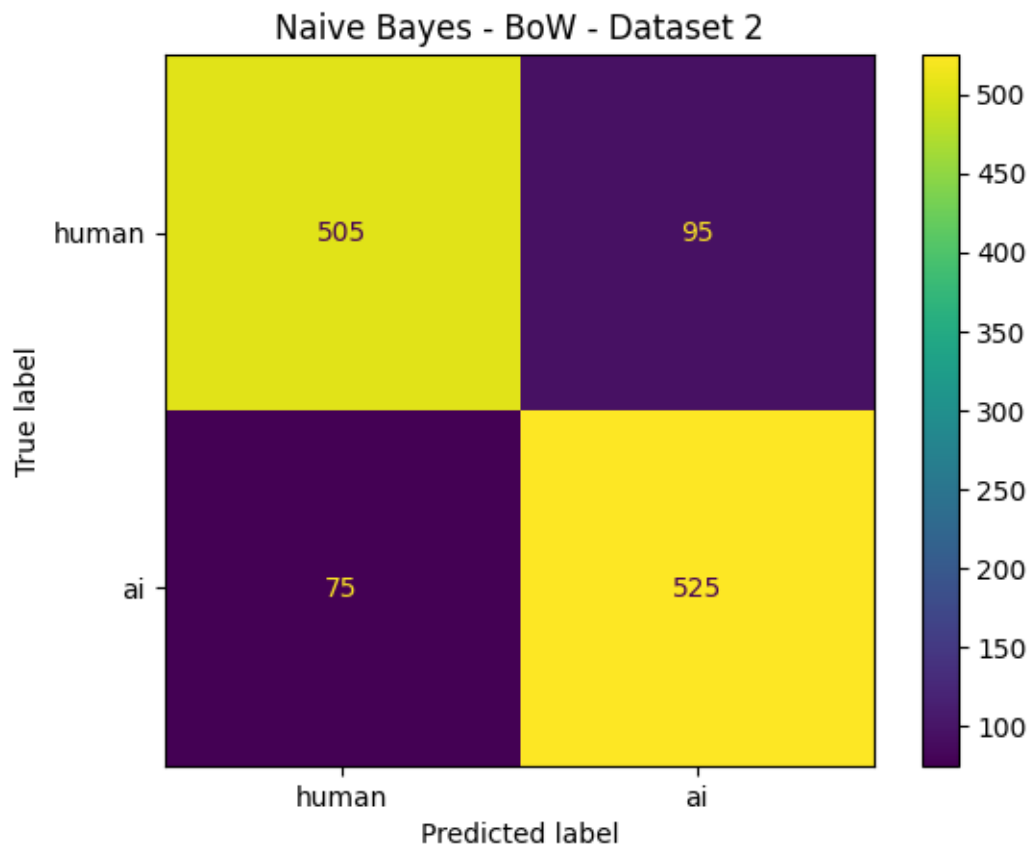
### **Sample output:**

[Type here]



[Type here]





Confusion matrices for Naive Bayes models displayed above.

[Type here]

## Code description:

The code defines a list of (representation, dataset) combinations, then loops through each pair to pull the stored predictions for Naive Bayes out of results, and plot each confusion matrix with ConfusionMatrixDisplay. Labels are simply human and ai. Every plot title gives us an idea of what it is we are viewing.

## ■ Logistic Regression

### Description:

This task trains and evaluates a Logistic Regression classifier across all four combinations of feature representation (BoW, TF-IDF) and dataset (Dataset 1 - DAIGT V2, Dataset 2 - HC3), storing the results for later comparison against other models.

### Code:

```
results[('Logistic Regression', 'BoW', 'Dataset 1')] =  
evaluate(LogisticRegression(max_iter=1000), X1_train_bow, y1_train,  
X1_test_bow, y1_test, 'Logistic Regression | BoW | Dataset 1 (DAIGT  
V2)')  
results[('Logistic Regression', 'TF-IDF', 'Dataset 1')] =  
evaluate(LogisticRegression(max_iter=1000), X1_train_tfidf, y1_train,  
X1_test_tfidf, y1_test, 'Logistic Regression | TF-IDF | Dataset 1  
(DAIGT V2)')  
results[('Logistic Regression', 'BoW', 'Dataset 2')] =  
evaluate(LogisticRegression(max_iter=1000), X2_train_bow, y2_train,  
X2_test_bow, y2_test, 'Logistic Regression | BoW | Dataset 2 (HC3)')  
results[('Logistic Regression', 'TF-IDF', 'Dataset 2')] =  
evaluate(LogisticRegression(max_iter=1000), X2_train_tfidf, y2_train,  
X2_test_tfidf, y2_test, 'Logistic Regression | TF-IDF | Dataset 2  
(HC3)')
```

### Sample output:

| Logistic Regression   BoW   Dataset 1 (DAIGT V2) |           |        |          |         |
|--------------------------------------------------|-----------|--------|----------|---------|
|                                                  | precision | recall | f1-score | support |
| 0                                                | 0.98      | 0.99   | 0.98     | 600     |
| 1                                                | 0.99      | 0.97   | 0.98     | 600     |
| accuracy                                         |           |        | 0.98     | 1200    |
| macro avg                                        | 0.98      | 0.98   | 0.98     | 1200    |
| weighted avg                                     | 0.98      | 0.98   | 0.98     | 1200    |

Accuracy: 0.9825

-----

| Logistic Regression   TF-IDF   Dataset 1 (DAIGT V2) |           |        |          |         |
|-----------------------------------------------------|-----------|--------|----------|---------|
|                                                     | precision | recall | f1-score | support |
| 0                                                   | 0.97      | 0.99   | 0.98     | 600     |
| 1                                                   | 0.99      | 0.96   | 0.98     | 600     |
| accuracy                                            |           |        | 0.98     | 1200    |

[Type here]

|              |      |      |      |      |
|--------------|------|------|------|------|
| macro avg    | 0.98 | 0.98 | 0.98 | 1200 |
| weighted avg | 0.98 | 0.98 | 0.98 | 1200 |

Accuracy: 0.9758333333333333

---

| Logistic Regression   BoW   Dataset 2 (HC3) |           |        |          |         |
|---------------------------------------------|-----------|--------|----------|---------|
|                                             | precision | recall | f1-score | support |
| 0                                           | 0.91      | 0.96   | 0.94     | 600     |
| 1                                           | 0.96      | 0.91   | 0.93     | 600     |
| accuracy                                    |           |        | 0.93     | 1200    |
| macro avg                                   | 0.93      | 0.93   | 0.93     | 1200    |
| weighted avg                                | 0.93      | 0.93   | 0.93     | 1200    |

Accuracy: 0.9333333333333333

---

| Logistic Regression   TF-IDF   Dataset 2 (HC3) |           |        |          |         |
|------------------------------------------------|-----------|--------|----------|---------|
|                                                | precision | recall | f1-score | support |
| 0                                              | 0.89      | 0.90   | 0.89     | 600     |
| 1                                              | 0.90      | 0.89   | 0.89     | 600     |
| accuracy                                       |           |        | 0.89     | 1200    |
| macro avg                                      | 0.89      | 0.89   | 0.89     | 1200    |
| weighted avg                                   | 0.89      | 0.89   | 0.89     | 1200    |

Accuracy: 0.8925

---

Logistic Regression model evaluation results stored in the results dictionary.

## Code description:

As in Naive Bayes, except `LogisticRegression(max_iter=1000)` is used instead. Each combination is evaluated with the corresponding BoW or TF-IDF split, and the outcomes are recorded under a key that includes the algorithm, representation and dataset information.

### ■ Confusion matrices for Logistic Regression

#### Description:

This task produces the confusion matrices for the Logistic Regression model of the same four combinations of representation/dataset and allows to compare the error patterns (false positives vs. false negatives) of the Logistic Regression model with the other classifiers.

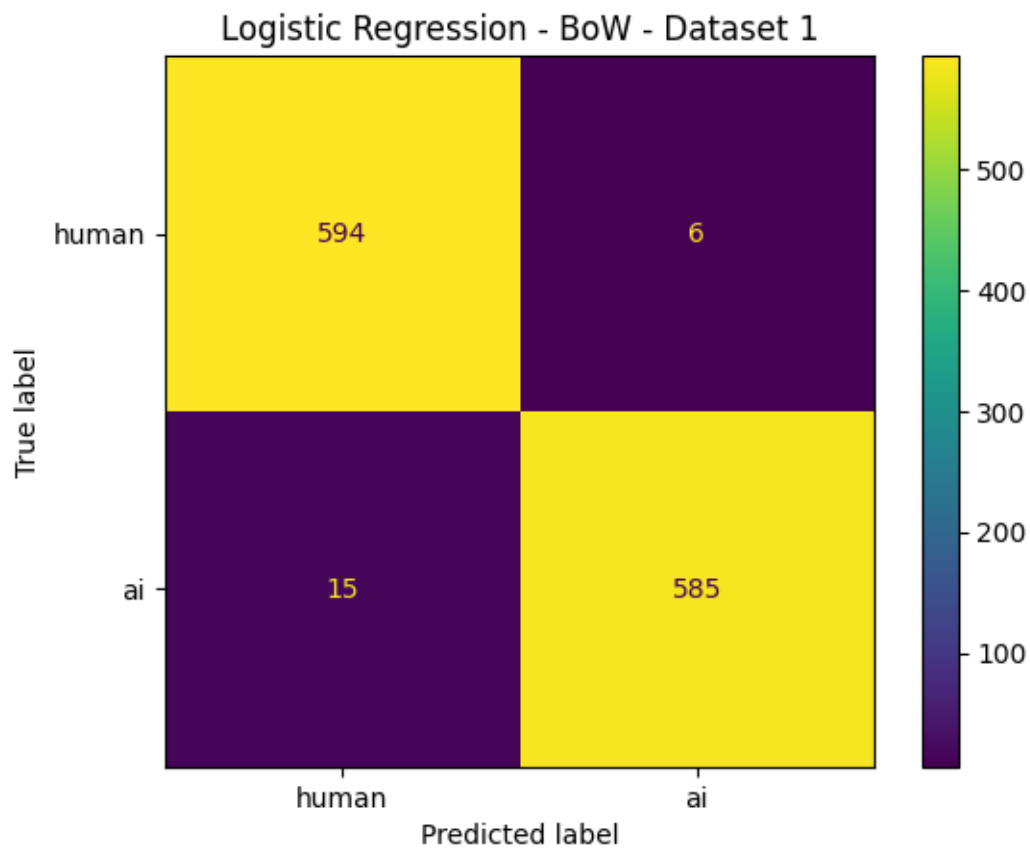
#### Code:

```
combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW',
'Dataset 2'), ('TF-IDF', 'Dataset 2')]
for rep, ds in combos:
    scores = results[('Logistic Regression', rep, ds)]
    ConfusionMatrixDisplay(confusion_matrix(scores['y_true'],
scores['y_pred']), display_labels=['human', 'ai']).plot()
    plt.title('Logistic Regression' + ' - ' + rep + ' - ' + ds)
    plt.show()
```

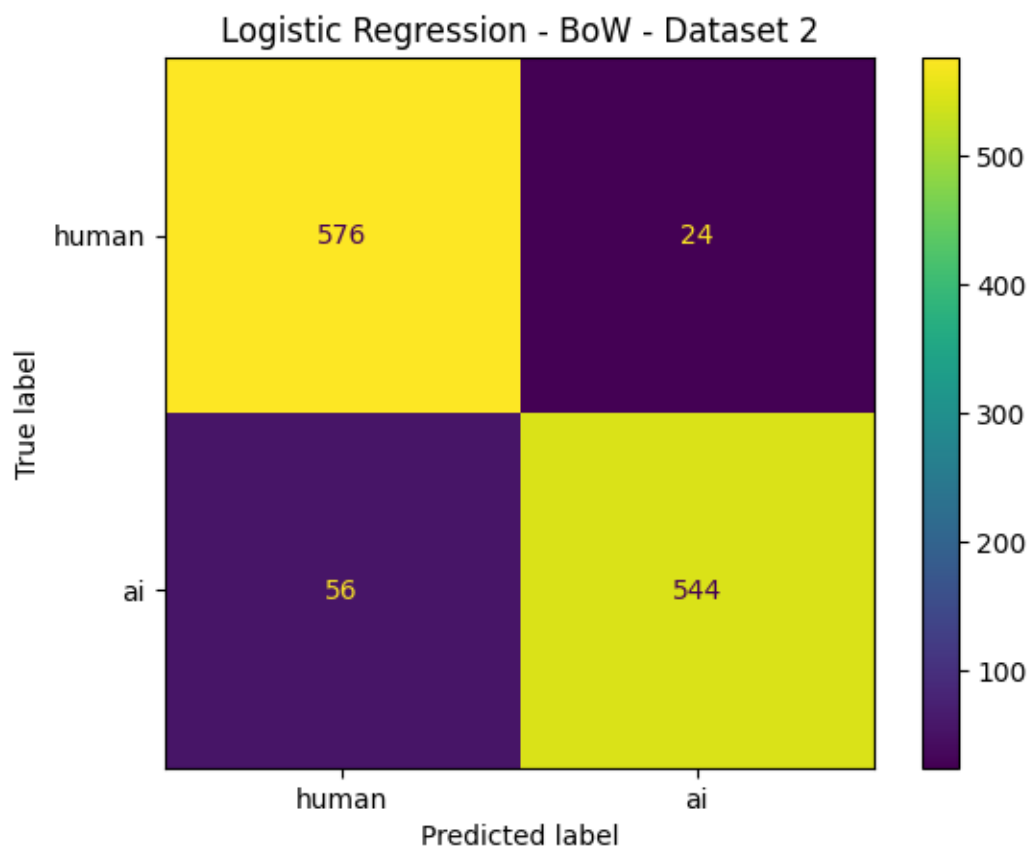
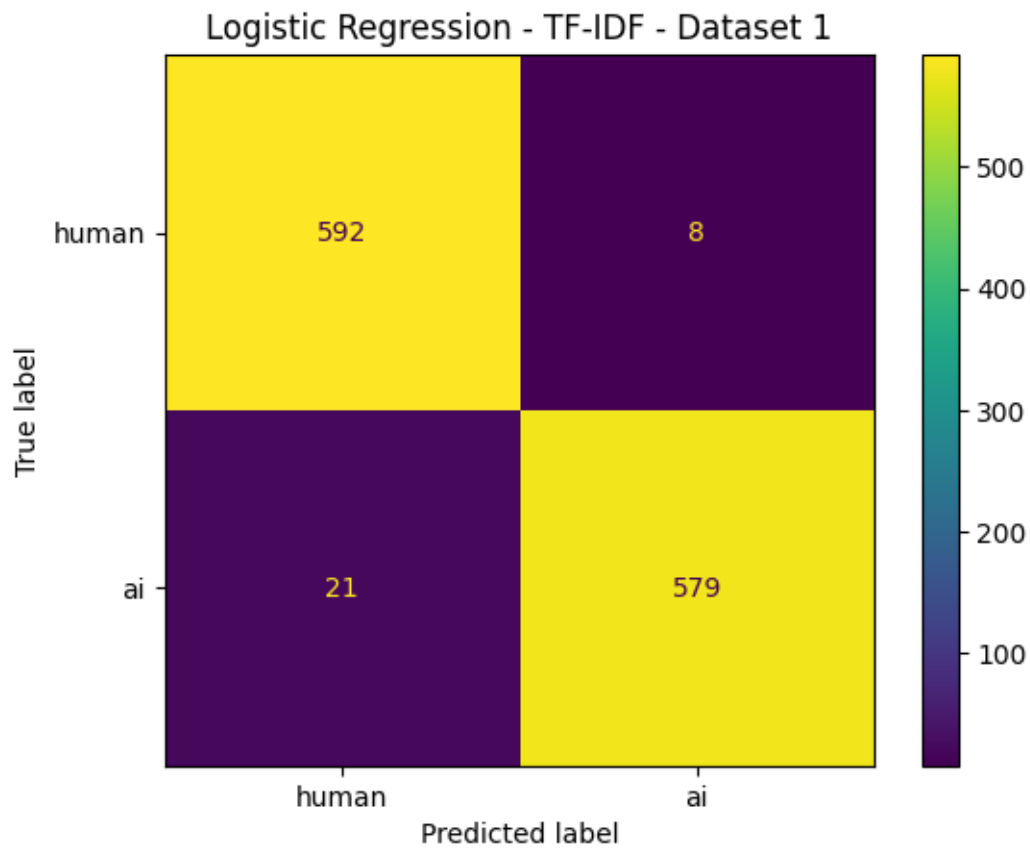
[Type here]

```
print('Confusion matrices for Logistic Regression models displayed  
above.')
```

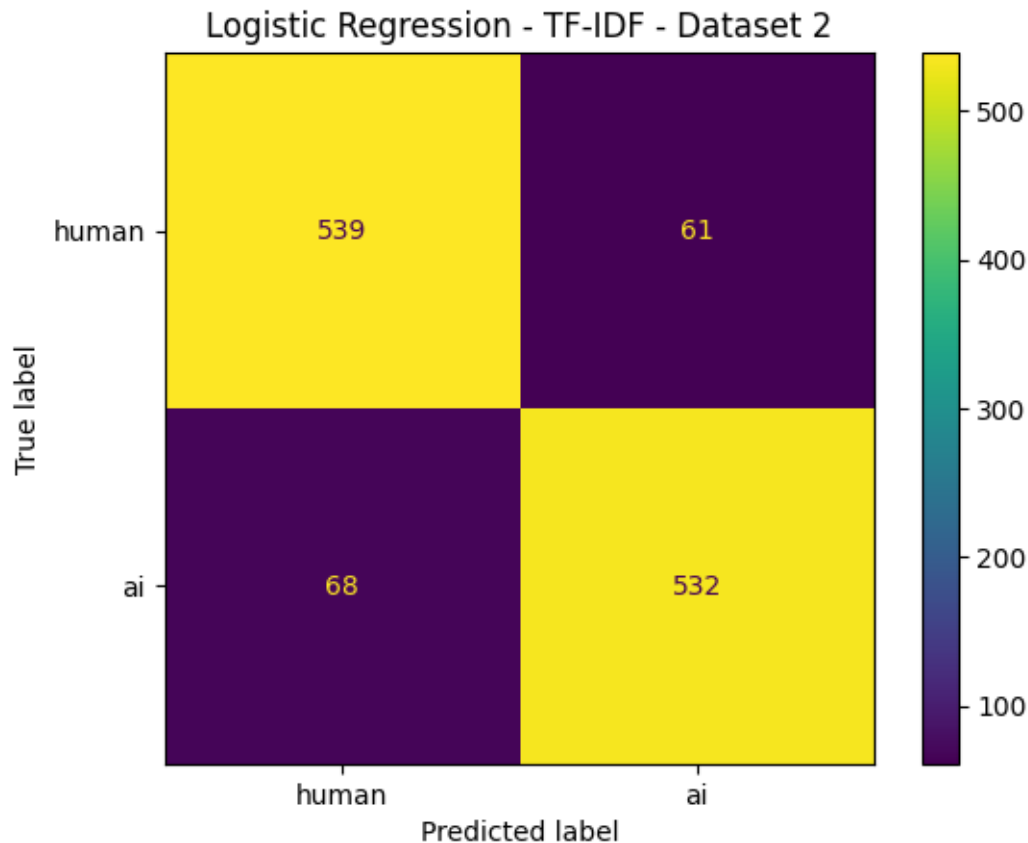
**Sample output:**



[Type here]



[Type here]



Confusion matrices for Logistic Regression models displayed above.

### Code description:

The same combination list is used to retrieve the Logistic Regression results for each (representation, dataset) pair, create a confusion matrix from the true and predicted labels, and use ConfusionMatrixDisplay to display the matrix. Before plots are displayed, the model, representation, and dataset names are written in the title on each plot.

#### ■ Support Vector Machine (LinearSVC)

##### Description:

Along with Naive Bayes and Logistic Regression, this task trains and evaluates a Linear Support Vector Machine (SVM) classifier using the same four combinations of representations/datasets..

##### Code:

```
results[('Support Vector Machine', 'BoW', 'Dataset 1')] =
evaluate(LinearSVC(max_iter=5000), X1_train_bow, y1_train, X1_test_bow,
y1_test, 'Support Vector Machine | BoW | Dataset 1 (DAIGT V2)')
results[('Support Vector Machine', 'TF-IDF', 'Dataset 1')] =
evaluate(LinearSVC(max_iter=5000), X1_train_tfidf, y1_train,
X1_test_tfidf, y1_test, 'Support Vector Machine | TF-IDF | Dataset 1
(DAIGT V2)')
```

[Type here]

```

results[('Support Vector Machine', 'BoW', 'Dataset 2')] =
evaluate(LinearSVC(max_iter=5000), X2_train_bow, y2_train, X2_test_bow,
y2_test, 'Support Vector Machine | BoW | Dataset 2 (HC3)')
results[('Support Vector Machine', 'TF-IDF', 'Dataset 2')] =
evaluate(LinearSVC(max_iter=5000), X2_train_tfidf, y2_train,
X2_test_tfidf, y2_test, 'Support Vector Machine | TF-IDF | Dataset 2
(HC3)')
print('Support Vector Machine model evaluation results stored in the
results dictionary.')

```

### Sample output:

```

Support Vector Machine | BoW | Dataset 1 (DAIGT V2)
      precision      recall  f1-score   support

      0      0.97      0.98      0.98      600
      1      0.98      0.97      0.98      600

    accuracy
  macro avg      0.98      0.98      0.98      1200
weighted avg      0.98      0.98      0.98      1200

```

Accuracy: 0.9775

```

-----
Support Vector Machine | TF-IDF | Dataset 1 (DAIGT V2)
      precision      recall  f1-score   support

      0      0.98      1.00      0.99      600
      1      1.00      0.98      0.99      600

    accuracy
  macro avg      0.99      0.99      0.99      1200
weighted avg      0.99      0.99      0.99      1200

```

Accuracy: 0.9875

```

-----
Support Vector Machine | BoW | Dataset 2 (HC3)
      precision      recall  f1-score   support

      0      0.89      0.94      0.91      600
      1      0.94      0.88      0.91      600

    accuracy
  macro avg      0.91      0.91      0.91      1200
weighted avg      0.91      0.91      0.91      1200

```

Accuracy: 0.9116666666666666

```

-----
Support Vector Machine | TF-IDF | Dataset 2 (HC3)
      precision      recall  f1-score   support

      0      0.91      0.90      0.91      600
      1      0.90      0.91      0.91      600

    accuracy
  macro avg      0.91      0.91      0.91      1200
weighted avg      0.91      0.91      0.91      1200

```

[Type here]

Accuracy: 0.9083333333333333

-----  
Support Vector Machine model evaluation results stored in the results dictionary.

### Code description:

The code creates a model (LinearSVC) for each combination and applies the model to the training/test features for each of the two datasets (DAIGT V2 and HC3) using the evaluate function. Each evaluation result is stored in the results dictionary, with a descriptive label to indicate each specific evaluation run and with keys of model, representation, and dataset.

### ■ Confusion matrices for Support Vector Machine

#### Description:

This task generates the four sets of confusion matrices for the Support Vector Machine model, and is part of a series of visual diagnostics required to compare the classification behavior for all three models and both feature representations/datasets.

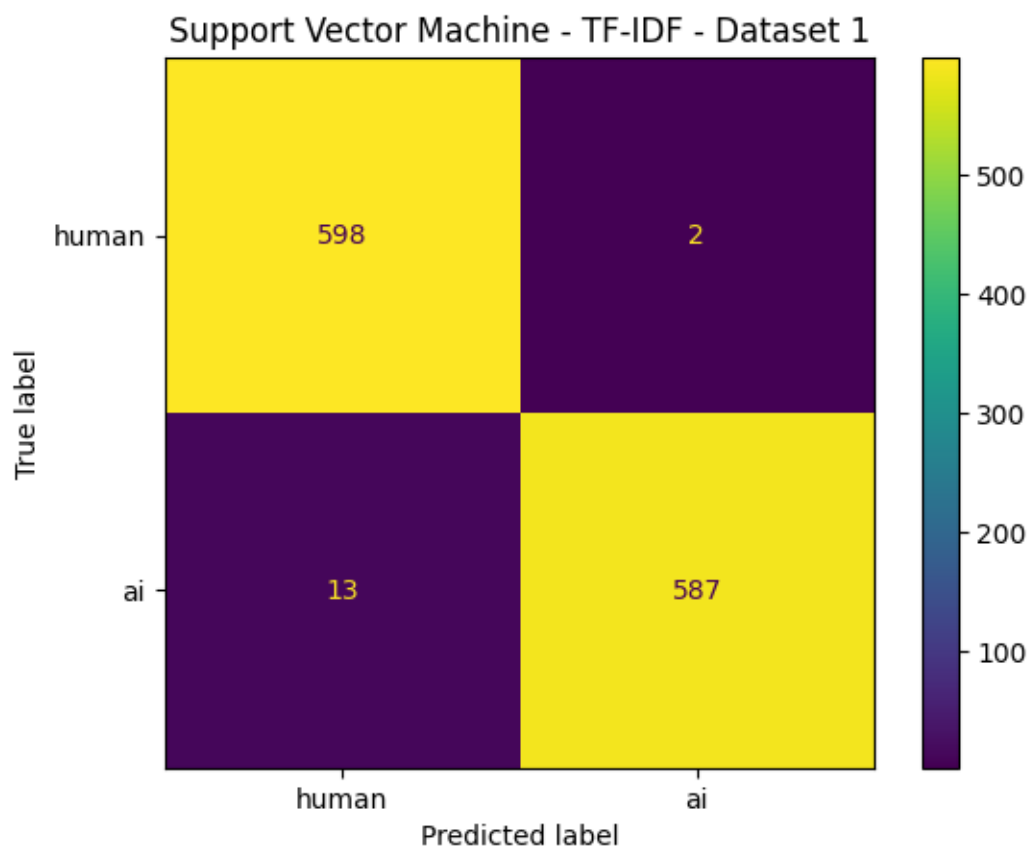
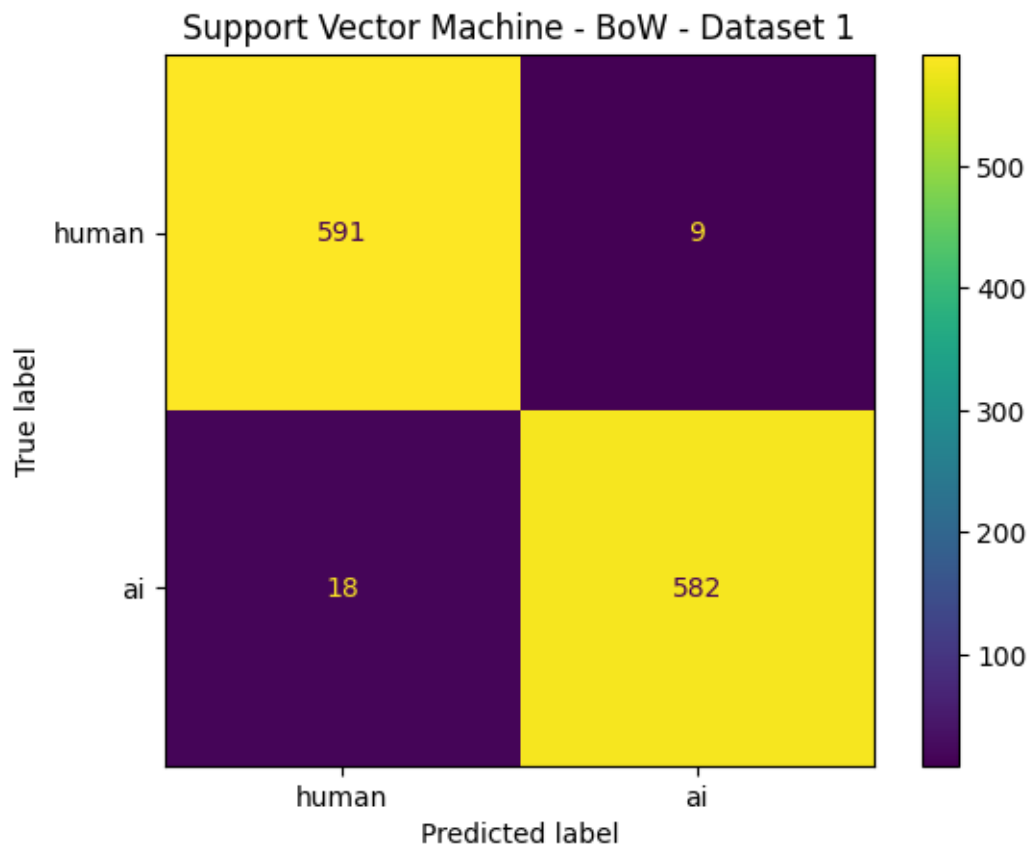
#### Code:

```
combos = [('BoW', 'Dataset 1'), ('TF-IDF', 'Dataset 1'), ('BoW',  
'Dataset 2'), ('TF-IDF', 'Dataset 2')]  
for rep, ds in combos:  
    scores = results[('Support Vector Machine', rep, ds)]  
    ConfusionMatrixDisplay(confusion_matrix(scores['y_true'],  
scores['y_pred']), display_labels=['human', 'ai']).plot()  
    plt.title('Support Vector Machine' + ' - ' + rep + ' - ' + ds)  
    plt.show()  
print('Confusion matrices for Support Vector Machine models displayed  
above.')
```

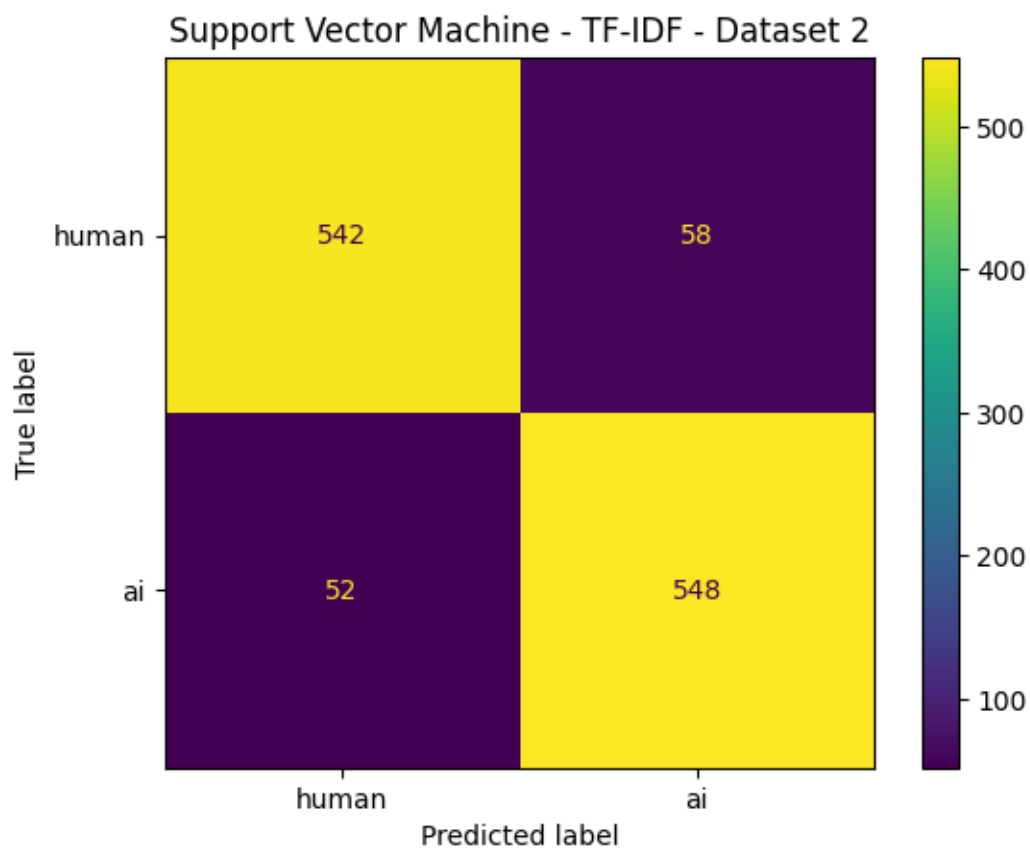
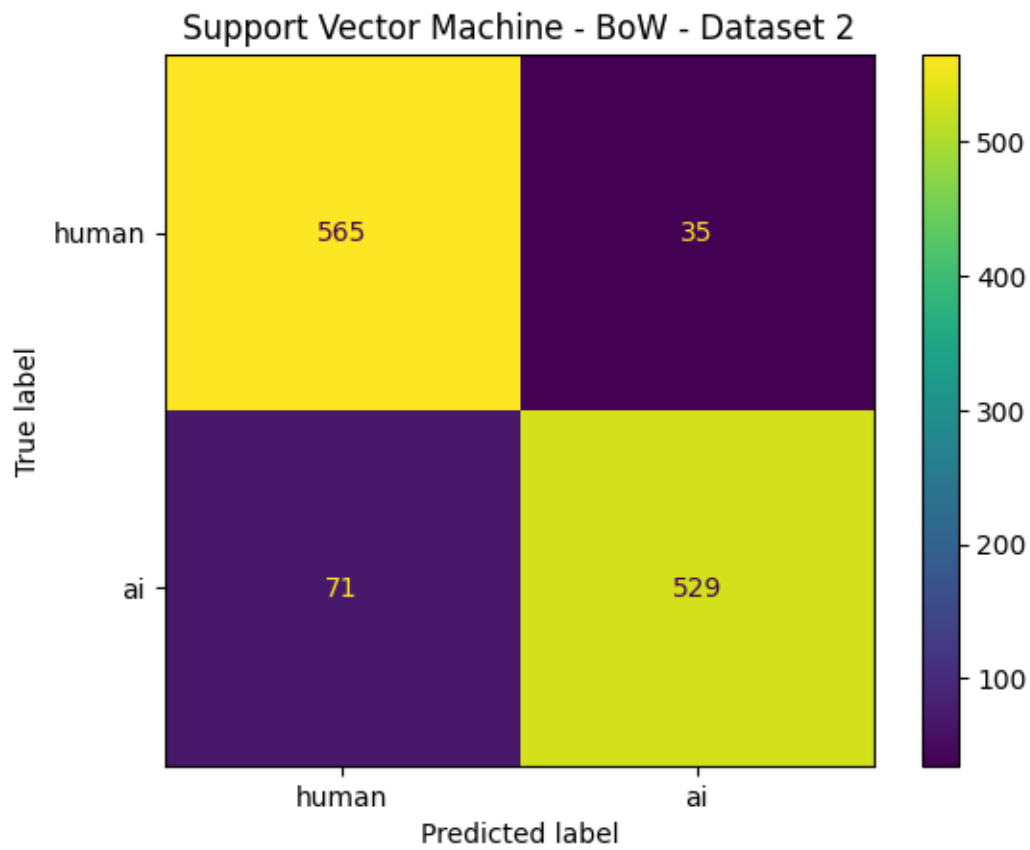
#### Sample output:

[Type here]





[Type here]



Confusion matrices for Support Vector Machine models displayed above.

[Type here]

## Code description:

The code repeats the same combinations and for each (representation, dataset) pair, it gets the true and predicted labels from the results of the SVM model. It calculates and plots the confusion matrix with labelled classes (human, ai), labels each figure, and prints a summary message after the 4 plots are shown.

### ■ Detailed metrics per algorithm (Accuracy, Precision, Recall, F1)

## Description:

Accuracy alone can be misleading so we also reported precision, recall and F1 for each and every algorithm. That becomes 16 numbers per model once you break it down by representation and dataset.

## Code:

```
daigt_raw = pd.read_csv(path_daigt_download)
df1 = daigt_raw[['text', 'label']].dropna()
df1['label'] = df1['label'].astype(int)
print("=" * 60)
print("Dataset 1: DAIGT V2 Train Dataset")
print("Dataset Summary")
print("=" * 60)
print(f"Total Samples : {len(df1):,}")
print(f"Total Columns : {df1.shape[1]}")
print("\nClass Distribution")
print(df1['label']
      .value_counts()
      .sort_index()
      .rename_axis("Label")
      .reset_index(name="Number of Samples"))
print("\nFirst 5 Records")
print(df1.head())
```

## Sample output:

Naive Bayes

| Representation | Dataset   | Accuracy | Precision | Recall | F1     |
|----------------|-----------|----------|-----------|--------|--------|
| BoW            | Dataset 1 | 0.9567   | 0.9580    | 0.9567 | 0.9566 |
| TF-IDF         | Dataset 1 | 0.9508   | 0.9515    | 0.9508 | 0.9508 |
| BoW            | Dataset 2 | 0.8583   | 0.8587    | 0.8583 | 0.8583 |
| TF-IDF         | Dataset 2 | 0.8275   | 0.8387    | 0.8275 | 0.8261 |

=====

Logistic Regression

| Representation | Dataset   | Accuracy | Precision | Recall | F1     |
|----------------|-----------|----------|-----------|--------|--------|
| BoW            | Dataset 1 | 0.9825   | 0.9826    | 0.9825 | 0.9825 |
| TF-IDF         | Dataset 1 | 0.9758   | 0.9761    | 0.9758 | 0.9758 |
| BoW            | Dataset 2 | 0.9333   | 0.9346    | 0.9333 | 0.9333 |
| TF-IDF         | Dataset 2 | 0.8925   | 0.8926    | 0.8925 | 0.8925 |

=====

Support Vector Machine

| Representation | Dataset   | Accuracy | Precision | Recall | F1     |
|----------------|-----------|----------|-----------|--------|--------|
| BoW            | Dataset 1 | 0.9775   | 0.9776    | 0.9775 | 0.9775 |
| TF-IDF         | Dataset 1 | 0.9875   | 0.9877    | 0.9875 | 0.9875 |
| BoW            | Dataset 2 | 0.9117   | 0.9132    | 0.9117 | 0.9116 |

[Type here]

```

TF-IDF Dataset 2    0.9083    0.9084    0.9083    0.9083
=====

```

### Code description:

For each algorithm we loop through its four stored results and print them together as one table. Since our classes are balanced, weighted and macro averages come out identical here.

### ■ Final Result Table [Accuracy]

#### Description:

This is the summary table the project brief asked for: three algorithms and two representations down the rows and both datasets across the columns.

#### Code:

```

rows = []
for algo in ['Naive Bayes', 'Logistic Regression', 'Support Vector
Machine']:
    for rep in ['BoW', 'TF-IDF']:
        rows.append({
            'Algorithm': algo,
            'Representation': rep,
            'Dataset 1': round(results[(algo, rep, 'Dataset
1')]['Accuracy'], 4),
            'Dataset 2': round(results[(algo, rep, 'Dataset
2')]['Accuracy'], 4)
        })

final_table = pd.DataFrame(rows)
print(final_table.to_string(index=False))

```

#### Sample output:

| Algorithm              | Representation | Dataset 1 | Dataset 2 |
|------------------------|----------------|-----------|-----------|
| Naive Bayes            | BoW            | 0.9567    | 0.8583    |
| Naive Bayes            | TF-IDF         | 0.9508    | 0.8275    |
| Logistic Regression    | BoW            | 0.9825    | 0.9333    |
| Logistic Regression    | TF-IDF         | 0.9758    | 0.8925    |
| Support Vector Machine | BoW            | 0.9775    | 0.9117    |
| Support Vector Machine | TF-IDF         | 0.9875    | 0.9083    |

### Code description:

Two nested loops walk through our three algorithms and two representations, pulling each accuracy score out of results and which assembles everything into a DataFrame.

[Type here]

## Result Table Discussion

| Algorithm              | Representation | Dataset 1 | Dataset 2 |
|------------------------|----------------|-----------|-----------|
| Naive Bayes            | Bow            | 0.9567    | 0.8583    |
|                        | TF-IDF         | 0.9508    | 0.8275    |
| Logistic Regression    | Bow            | 0.9825    | 0.9333    |
|                        | TF-IDF         | 0.9758    | 0.8925    |
| Support Vector Machine | Bow            | 0.9775    | 0.9117    |
|                        | TF-IDF         | 0.9875    | 0.9083    |

### Description:

Logistic Regression and the Support Vector Machine outperformed Naive Bayes on both data sets, and it's not by a little bit. Our worst performance is with Naive Bayes on HC3, at 0.8275, sixteen points below the best score on DAIGT. Honestly, we were surprised by how much larger the difference between datasets was compared to the difference between algorithms. All of the models pass the DAIGT at 0.95. None of them have an HC3 of 0.94. We believe this is because the two corpora are quite different in terms of their form: The AI essays written by the DAIGT are written from a small set of prompts, and hence do not vary much from one to the next, while the HC3 answers are shorter and drawn from five different domains and vary considerably from one to the next — and this provides the models with less of a pattern to attach themselves to. The best DAIGT we achieved was Support Vector Machine with TF-IDF (0.9875). Logistic Regression with Bag of Words gave the best HC3 result with 0.9333. For this task, the gap remained less than a point between BoW and TF-IDF for both the linear models, so the choice of the classifiers was more important than the representation. We have to be honest about one thing: the numbers are derived from a balanced sample of 6,000 rows from each of the datasets, not from the corpora; and each test set was drawn from the same distribution as its training set. We cannot tell from this how they would perform on text that they haven't seen before from a generator.

[Type here]

